

STOCK PRICE PREDICTION USING MACHINE LEARNING

A PROJECT REPORT

Submitted by

S.JANESH KUMAR (211421104104)

W.KEN BOSTON (211421104126)

P.MALAI RAJA (211421104152)

*in partial fulfilment for the award of the degree
of*

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

PANIMALAR ENGINEERING COLLEGE, CHENNAI 600 123

ANNA UNIVERSITY, CHENNAI 600 025

APRIL 2025



STOCK PRICE PREDICTION USING MACHINE LEARNING

A PROJECT REPORT

Submitted by

S.JANESH KUMAR (211421104104)

W.KEN BOSTON (211421104126)

P.MALAI RAJA(211421104152)

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

PANIMALAR ENGINEERING COLLEGE, CHENNAI 600 123

ANNA UNIVERSITY, CHENNAI 600 025

APRIL 2025

BONAFIDE CERTIFICATE

Certified that this project report “STOCK PRICE PREDICTION USING MACHINE LEARNING” is the bonafide work of “S.JANESH KUMAR (211421104104),W.KENBOSTON(211421104126),P.MALAIRAJA (211421104152)” who carried out the project work under my supervision.

SIGNATURE

**Dr. L. JABASHEELA, M.E., Ph.D.,
HEAD OF THE DEPARTMENT**

DEPARTMENT OF CSE,
PANIMALAR ENGINEERING COLLEGE,
NASARATHPETTAI,
POONAMALLEE,
CHENNAI - 600 123.

SIGNATURE

**Dr.V.SATHIYA,M.E,Phd
SUPERVISOR**

DEPARTMENT OF CSE,
PANIMALAR ENGINEERING COLLEGE,
NASARATHPETTAI,
POONAMALLEE,
CHENNAI - 600 123.

Certified that the above candidate(s) were examined in the Anna University Project Viva-Voce Examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION BY THE STUDENT

We **S.JANESH KUMAR (211421104104), W.KENBOSTON(211421104126), P.MALAI RAJA(211421104152)** hereby declare that this project report titled “STOCK PRICE PREDICTION USING MACHINE LEARNING”, under the guidance of **Dr.V.SATHIYA , SUPERVISOR** is the original work done by us and we have not plagiarized or submitted to any other degree in any university by us.

ACKNOWLEDGEMENT

We would like to express our deep gratitude to our respected Secretary and Correspondent **Dr. P. CHINNADURAI, M.A., Ph.D** for his kind words and enthusiastic motivation, which inspired us a lot in completing this project.

We express our sincere thanks to our Directors **Tmt. C. VIJAYARAJESWARI, Dr. C. SAKTHI KUMAR, M.E., Ph. D** and **Dr. SARANYASREE SAKTHI KUMAR B.E., M.B.A., Ph.D** for providing us with the necessary facilities to undertake this project.

We also express our gratitude to our Principal **Dr. K. Mani, M.E., Ph.D** who facilitated us in completing the project.

We thank the Head of the CSE Department, **Dr. L. JABASHEELA, M.E., Ph.D** for the support extended throughout the project.

We would like to thank our Project Guide **Dr.V.SATHIYA** , and all the faculty members of the Department of CSE for their advice and encouragement for the successful completion of the project.

S.JANESH KUMAR

W.KEN BOSTON

P.MALAI RAJA

ABSTRACT

Stock price prediction is a challenging task due to the volatile and complex nature of financial markets. Various approaches, including statistical methods, machine learning, and deep learning, have been employed to improve prediction accuracy. This study explores different techniques such as time series analysis, regression models, and neural networks, including Long Short-Term Memory (LSTM) networks, to forecast stock prices based on historical data and market indicators. Factors such as trading volume, economic trends, and sentiment analysis from news and social media are also considered. The findings suggest that advanced machine learning models outperform traditional methods in capturing market trends and patterns, providing investors with valuable insights for decision-making. Stock price prediction is a complex and dynamic problem influenced by various factors such as historical price trends, market sentiment, macroeconomic indicators, and company performance. Traditional methods like statistical time series models (ARIMA, GARCH) have been widely used for forecasting but often fail to capture the non-linear and highly volatile nature of stock prices. In recent years, machine learning and deep learning techniques, including Support Vector Machines (SVM), Random Forest, and neural networks like Long Short-Term Memory (LSTM) and Transformer models, have gained popularity due to their ability to recognize patterns and process large datasets efficiently. Additionally, sentiment analysis from news articles, social media, and financial reports further enhances predictive accuracy by incorporating external market influences..

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	v
	List Of Tables	vi
	List of Figures	vii
	List of Abbreviations	viii
1.	INTRODUCTION	1
	1.1 Problem Definition	3
2.	LITERATURE REVIEW	5
3.	THEORETICAL BACKGROUND	8
	3.1 Implementation Environment	9
	3.2 System Architecture	10
	3.3 Proposed Methodology	11
	3.3.1 Database Design/Data Set Description	12
	3.3.2 Input Design(UI)	13
	3.3.3 Module Design	14
4.	SYSTEM IMPLEMENTATION	15
	4.1 Algorithm1/Module 1 Name	16
	4.2 Algorithm2/Module 2 Name	19
5.	RESULTS AND DISCUSSION	20
	5.1 Performace Parameters- Testing	21
	5.2 Results and Discussion	23
6 .	CONCLUSION AND FUTURE WORK	24
	APPENDICES	25
	A1.SDG GOALS	26
	A2.Source Code	27

CHAPTER NO	TITLE	PAGE NO
6.	A.3 Screenshots	35
	A.4 Plagiarism Report	51
	A.4 Paper Publication	41
	REFERENCES	43

LIST OF FIGURES

Figure No.	Figure Name
4.1.	Activity diagram
4.2.	Class Diagram
4.3	Sequence Diagram
4.4	Entity Relationship Diagram
4.5	Data Flow Diagram
5.1.	Architecture Diagram
A.1	Outputs

LIST OF ABBREVIATIONS

S. No.	SYMBOL	ABBREVIATION
1.	CNN	Convolutional Neural Network
2.	GUI	Graphical User Interface
3.	.h5	Hierarchical Data Format version 5 (HDF5)
4.	KNN	K-Nearest Neighbors
5.	Adam	Adaptive Moment Estimation (Optimizer)
6.	ReLU	Rectified Linear Unit (Activation Function)
7.	LR	Learning Rate
8.	ML	Machine Learning

CHAPTER 1

INTRODUCTION

1. INTRODUCTION

Stock market prediction has been a widely researched topic due to its financial significance and impact on global economies. Investors, traders, and financial analysts constantly seek methods to forecast stock price movements to maximize returns and minimize risks. Traditional forecasting techniques, such as fundamental and technical analysis, rely on historical price trends, company performance, and economic indicators. However, these methods often struggle to capture the highly volatile and nonlinear nature of financial markets. With advancements in artificial intelligence and data-driven technologies, machine learning (ML) has emerged as a powerful tool for stock price prediction. ML algorithms can process vast amounts of historical stock data, identify hidden patterns, and make predictions based on learned relationships. Techniques such as Support Vector Machines (SVM), Decision Trees, Random Forests, and deep learning models like Artificial Neural Networks (ANNs) and Long Short-Term Memory (LSTM) networks have been successfully applied in financial forecasting. Moreover, integrating alternative data sources, such as news sentiment analysis, social media trends, and macroeconomic indicators, has further improved prediction accuracy. While challenges such as data noise, market unpredictability, and overfitting remain, machine learning continues to evolve, offering more sophisticated models for stock price forecasting. This study explores various ML techniques applied to stock market prediction, comparing their effectiveness and discussing their potential to enhance investment strategies.

1.1 PROBLEM DEFINITION

Stock price prediction is a complex and challenging task due to the highly volatile, dynamic, and non-linear nature of financial markets. Traditional forecasting techniques, such as statistical models and technical analysis, often fail to accurately predict market movements due to their inability to capture complex patterns and dependencies in stock price data. Investors and traders require more reliable and data-driven approaches to make informed decisions and minimize financial risks. The primary challenge in stock price prediction is the influence of multiple factors, including historical price trends, trading volume, macroeconomic indicators, news sentiment, and investor behavior. Additionally, external events such as economic policies, geopolitical tensions, and global crises further complicate market dynamics. The unpredictability of market fluctuations makes it difficult to develop a model that consistently delivers high-accuracy predictions. With the advancement of machine learning (ML) and deep learning techniques, there is a growing opportunity to leverage computational models that can process large datasets, identify hidden patterns, and improve predictive accuracy. However, several issues persist, such as data quality, feature selection, overfitting, and model interpretability. Therefore, this study aims to develop and compare machine learning models for stock price prediction, evaluating their effectiveness and limitations to provide investors with more reliable forecasting tools.

CHAPTER 2

LITERATURE

REVIEW

2. LITERATURE REVIEW

1. **Autoregressive Integrated Moving Average (ARIMA):** ARIMA models have been widely used for stock price forecasting due to their capability to capture trends and seasonality (Box & Jenkins, 1976). However, they struggle with highly volatile data. The AR part of ARIMA shows that the time series is regressed on its own past data. The MA part of ARIMA indicates that the forecast error is a linear combination of past respective errors. The *I* part of ARIMA shows that the data values have been replaced with differenced values of *d* order to obtain stationary data, which is the requirement of the ARIMA model approach. Why would one need to get to this level of complexity? The ARIMA model is effective in fitting past data with this combination approach and help forecast future points in a time series.
2. **Random Forests:** These ensemble learning models help in reducing overfitting and improving prediction accuracy by combining multiple decision trees (Patel et al., 2015). The accurate forecast or prediction of stock prices is a specially focused issue for the investors and companies listed in the stock market. Non-stationary and non-linear time-series nature of stock prices makes the prediction results very complex and challenging (Cavalcante et al., 2016). Financial time series analysis is a very important source of information for stock market prediction (Oztekin et al., 2016). Finding hidden patterns is the requirement for analysis and prediction of the stock price actuations. The pre-

assumption as given by very famous hypothesis Random Walk (Malkiel, B. G., 2003, Mankiw & Shapiro, 1985) and the Efficient Market (Jensen M. C., 1978) are stating that it is impossible to predict the nature of the stock market due to presence of randomness and nonlinearity in the dataset. These assumptions were verified by many different pursuance models in different interval of time (Atsalakis & Valavanis, 2009).

3. **Natural Language Processing (NLP):** Sentiment analysis using NLP techniques on financial news, earnings reports, and social media data (e.g., Twitter, Reddit) has been integrated into stock prediction models (Bollen et al., 2011). NLP enables computers and digital devices to recognize, understand and generate text and speech by combining computational linguistics—the rule-based modeling of human language—together with statistical modeling, machine learning and [deep learning](#). NLP research has helped enable the era of [generative AI](#), from the communication skills of (LLMs) to the ability of image generation models to understand requests. NLP is already part of everyday life for many, powering search engines, prompting [chatbots](#) for customer service with spoken commands, voice-operated GPS systems and question-answering digital assistants on smartphones such as Amazon's Alexa, Apple's Siri and Microsoft's Cortana.
4. **Unsupervised Learning:** Clustering techniques, such as K-Means and hierarchical clustering, have been used to group stocks with similar price movements for better prediction (Kou et al., 2021). The most common clustering covered in **machine learning for beginners** is K-Means. The first step is to create c new observations among our

unlabelled data and locate them randomly, called centroids. The number of centroids represents the number of output classes. The first step of the iterative process for each centroid is to find the nearest point (in terms of Euclidean distance) and assign them to its category. Next, for each category, the average of all the points attributed to that class is computed. The output is the new centroid of the class.

5. CNNs have been adapted to analyze stock price data and financial news sentiment, improving predictive power (Chen et al., 2019). [Neural networks](#) are a subset of machine learning, and they are at the heart of deep learning algorithms. They are comprised of node layers, containing an input layer, one or more hidden layers, and an output layer. Each node connects to another and has an associated weight and threshold. If the output of any individual node is above the specified threshold value, that node is activated, sending data to the next layer of the network. Otherwise, no data is passed along to the next layer of the network.

CHAPTER 3

THEORETICAL

BACKGROUND

3.THEORETICAL BACKGROUND

3.1IMPLEMENTATION ENVIRONMENT

The implementation environment for stock price prediction depends on the techniques and tools used:

1. Programming Languages

- **Python**
- **Javascript**

2. Development Environment & IDEs

- **Jupyter Notebook (For interactive development)**
- **VS Code**

3. Data Sources & APIs

- **Yahoo Finance API**
- **Alpha Vantage**
- **Quandl**
- **IEX Cloud**

HARDWARE REQUIREMENT

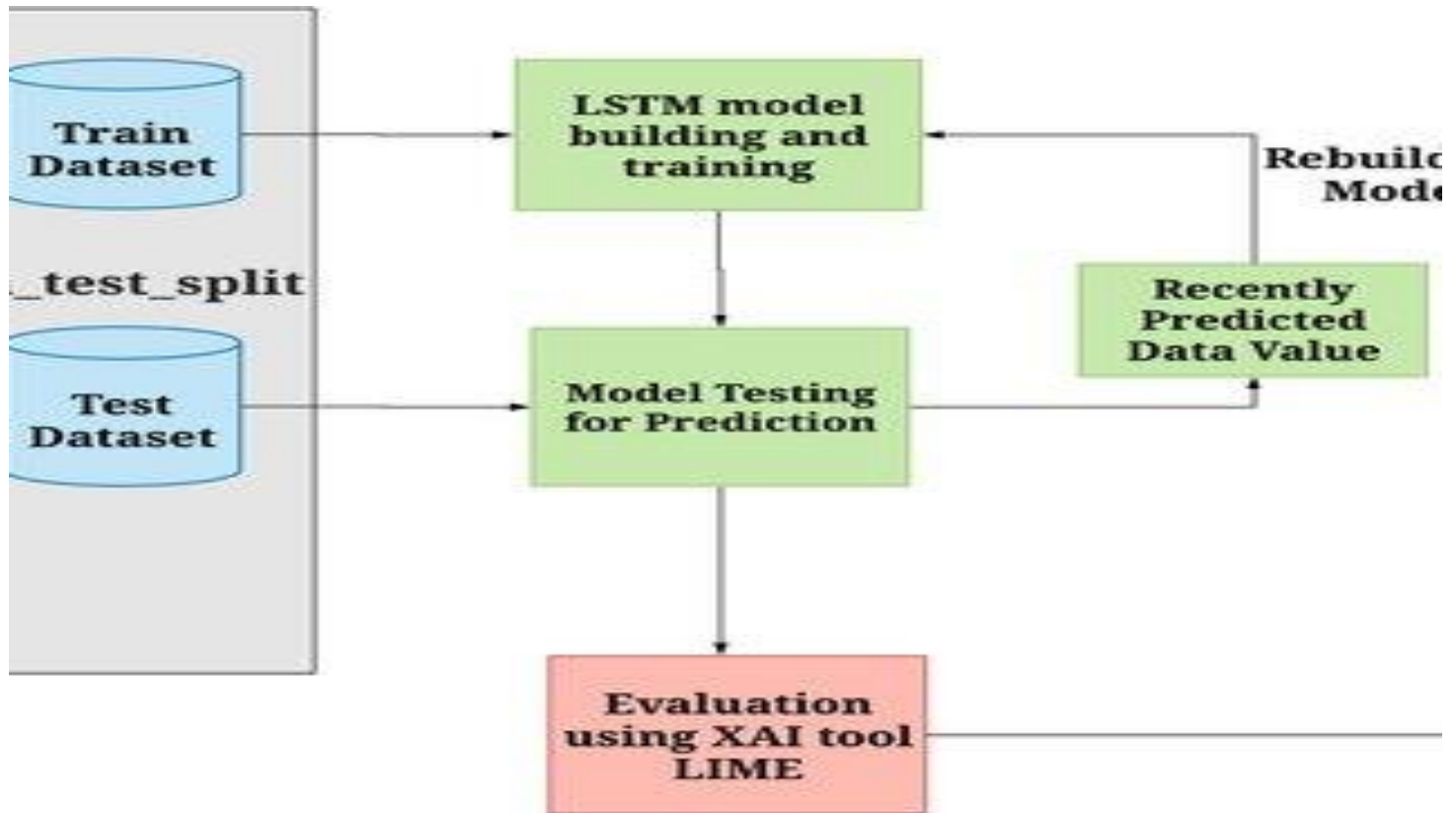
- **Hard Disk : 500GB and Above**
- **RAM : 4GB and Above**
- **Processor : i3 and Above**

SOFTWARE REQUIREMENT

- **Frontend : CSS,JavaScript,Html**
- **Backend : Python, Machine Learning**
- **Operating System : Windows 10 (64 bit)**
- **Software : Python 3.7**

- Tools : Visual Studio, Anaconda
- Framework : Flask

3.2 SYSTEM ARCHITECTURE



The system architecture for stock price prediction typically consists of multiple layers, including data collection, preprocessing, model training, prediction, and evaluation. The process begins with data collection, where historical stock prices, financial indicators, news sentiment, and other relevant data sources are gathered from APIs, databases, or web scraping. Next, the preprocessing layer cleans and normalizes the data, handling missing values, removing outliers, and transforming features to improve model performance. In the feature engineering stage, technical indicators (e.g., moving averages, RSI, MACD), fundamental data (e.g., P/E ratio, earnings reports), and alternative data (e.g., social media sentiment, macroeconomic indicators) are extracted.

3.3 PROPOSED METHODOLOGY

The proposed system aims to build an advanced and integrated framework for stock price prediction that overcomes the limitations of existing systems. It combines state-of-the-art machine learning, deep learning, and sentiment analysis techniques with a robust preprocessing pipeline to improve prediction accuracy, scalability, and adaptability to dynamic market conditions. Handling missing or incomplete data using interpolation techniques. Normalization and scaling of stock price data to improve model performance. Noise reduction using moving averages or filtering techniques. Sentiment extraction from textual data using natural language processing (NLP) techniques like tokenization, stemming, and sentiment scoring. In the **feature engineering** phase, technical indicators such as moving averages, Bollinger Bands, and Relative Strength Index (RSI) are extracted, along with fundamental and sentiment-based features from financial reports and news. The **model selection** phase explores traditional machine learning models like linear regression, decision trees, and support vector machines, alongside advanced deep learning models such as LSTMs, GRUs, and transformer-based architectures that capture sequential dependencies in stock data. The model is then **trained and validated** using techniques like cross-validation and hyperparameter tuning to optimize performance. Once trained, the model's accuracy is assessed using metrics such as MAE, RMSE, R^2 , and directional accuracy. The final step is **deployment**, where the model is integrated into a real-time prediction system via APIs or trading platforms for practical use. To enhance robustness, periodic retraining and model monitoring mechanisms are implemented,

ensuring adaptability to dynamic market conditions.

3.3.1 DATABASE DESIGN

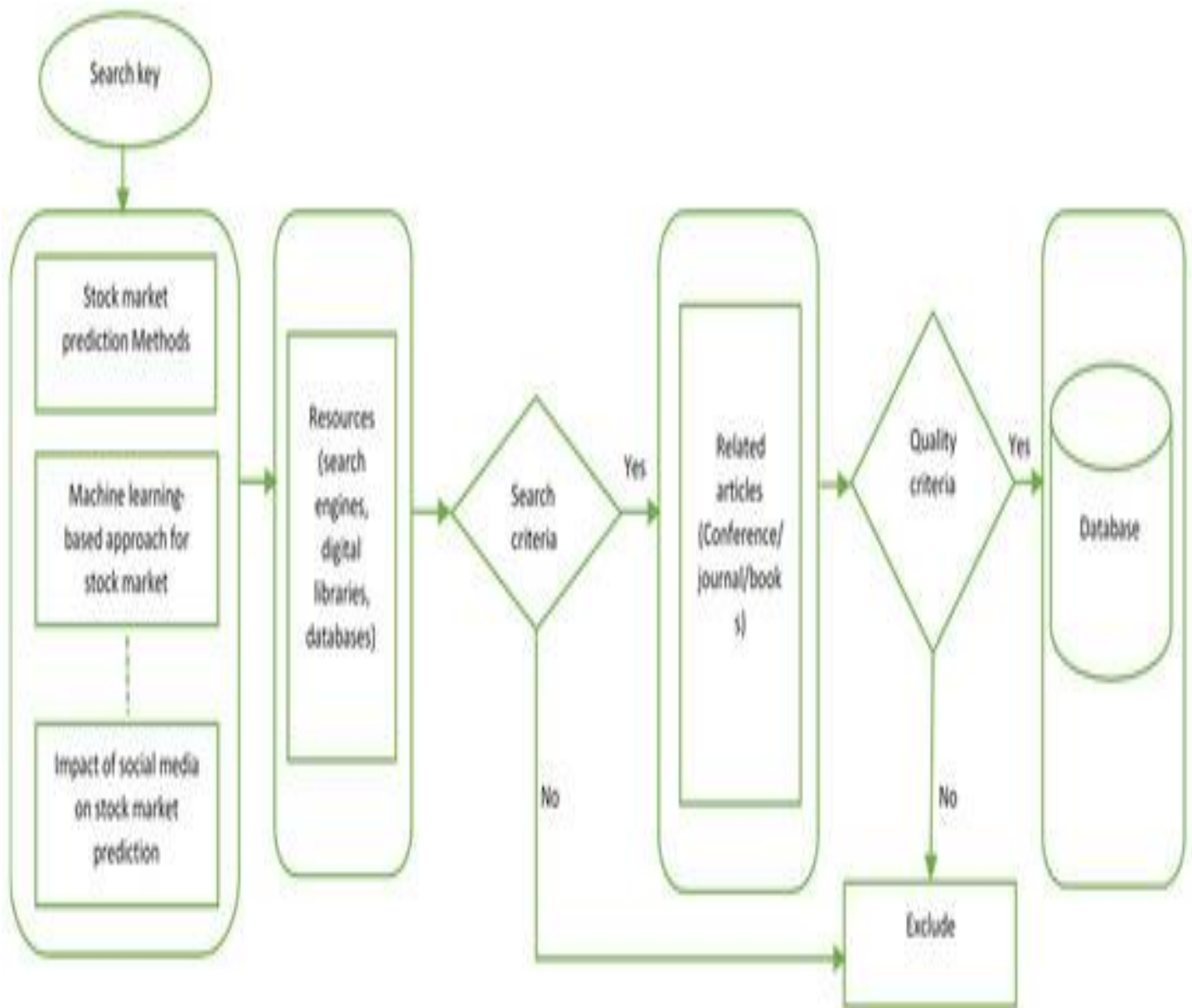
The database design for stock price prediction involves creating a structured schema that can efficiently store and manage a variety of data related to stock performance, market indicators, and related external factors. At the core, the design includes a **Stocks Table**, which holds basic information about each stock such as the `stock_id`, `ticker_symbol`, `company_name`, `sector`, `industry`, `market_cap`, and `ipo_date`. This table serves as a reference for all other data. A **historical Prices Table** tracks stock price data over time, linked to the Stocks Table through a foreign key (`stock_id`). This table stores essential fields such as `price_id`, `stock_id`, `date`, `open_price`, `close_price`, `high_price`, `low_price`, `volume`, and `adjusted_close_price`. For incorporating technical analysis, a **Technical Indicators Table** stores various calculated indicators such as moving averages (e.g., SMA, EMA), RSI, Bollinger Bands, etc., each linked to specific stock and date. A **News Sentiment Table** stores external news data, including `news_id`, `stock_id`, `date`, `headline`, `sentiment_score`, and `source`, capturing how news influences stock prices. Additionally, a **Model Predictions Table** stores predictions made by machine learning models, with fields like `prediction_id`, `stock_id`, `date`, `predicted_price`, `confidence_score`, and `model_version`, helping track the accuracy of different models over time. The database design supports integration with machine learning workflows, enabling efficient querying and real-time analysis while maintaining scalability for large datasets and historical data analysis.

3.3.2 INPUT DESIGN

The input design for stock price prediction is crucial for ensuring that the model receives the right data in an organized and efficient manner. The primary input sources include historical stock price data, market indicators, financial news, and external economic factors. For stock price prediction, **historical price data** serves as the backbone of input, consisting of features like opening price, closing price, high, low, volume, and adjusted close price, typically spanning multiple time periods (e.g., daily, weekly, monthly). **Technical indicators** such as moving averages (SMA, EMA), Relative Strength Index (RSI), and MACD are also important inputs, as they help capture trends and momentum in the stock price movement. **Market sentiment** derived from financial news articles, social media, or analysts' reports, can be included in the input as sentiment scores, which reflect public perception or investor sentiment about a particular stock or sector. **Macroeconomic indicators** like interest rates, inflation rates, GDP growth, and other relevant economic factors provide external context that can influence stock prices

3.3.3 MODULE DESIGN

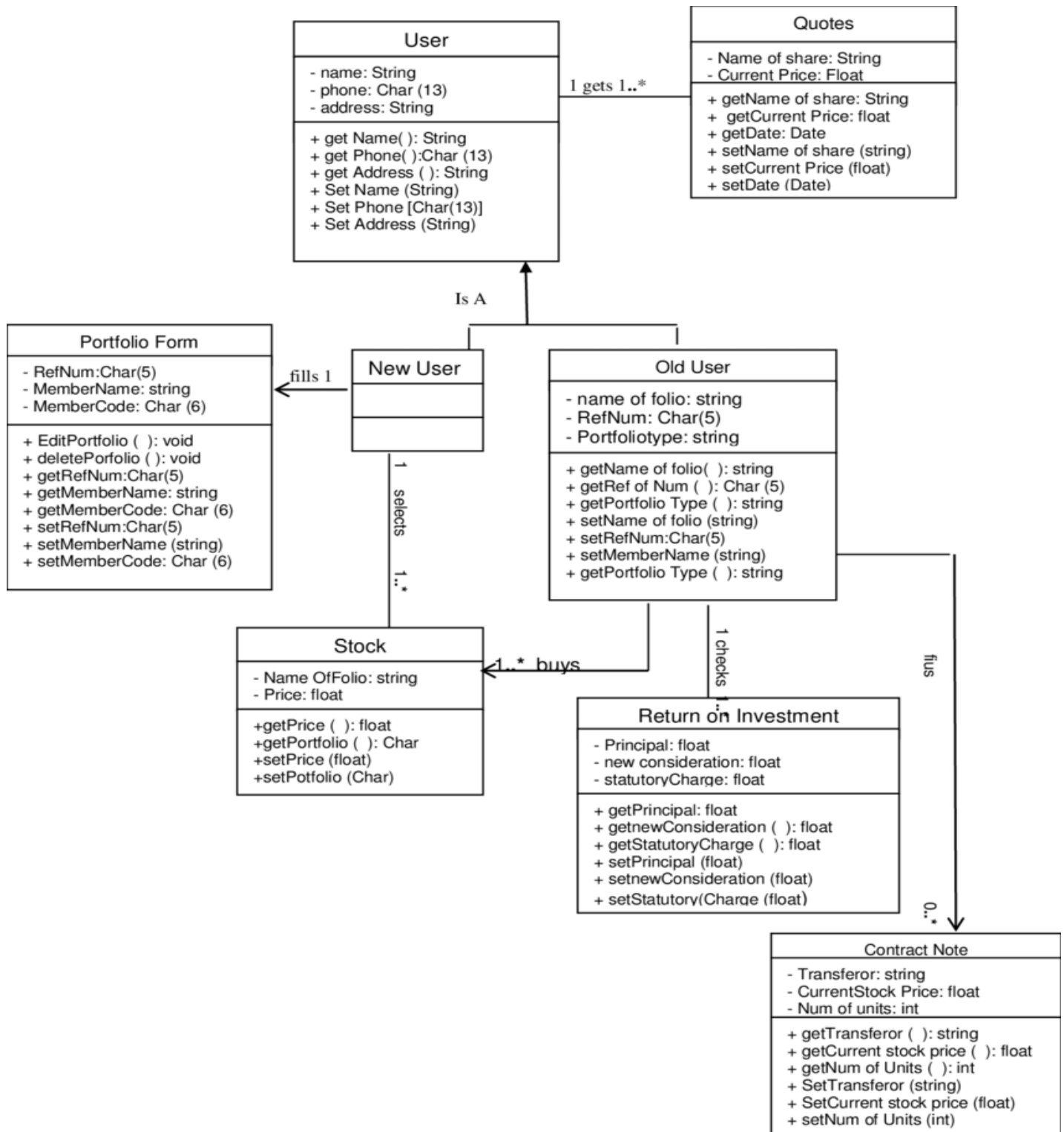
DATA FLOW DIAGRAM



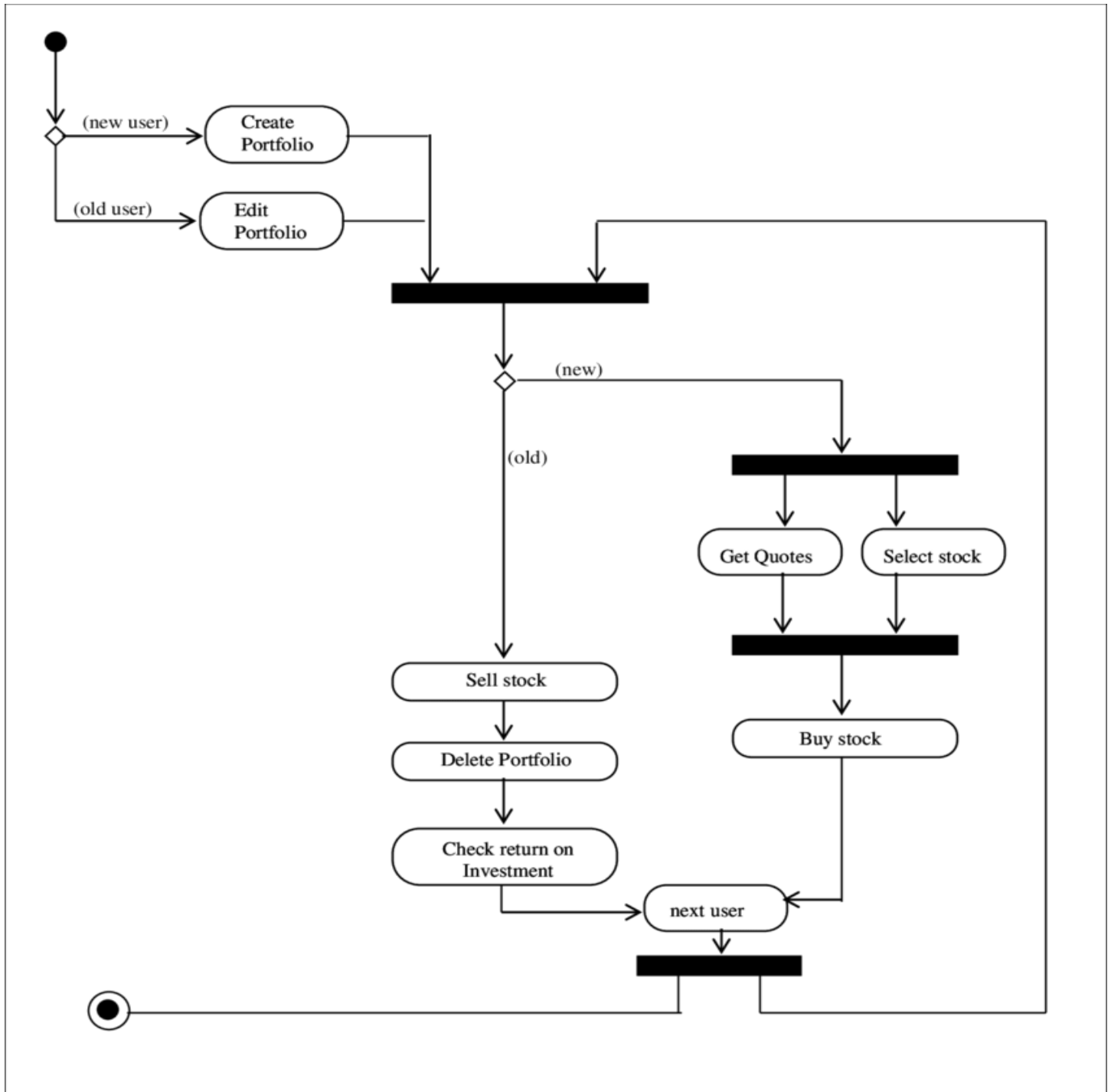
USE CASE DIAGRAM



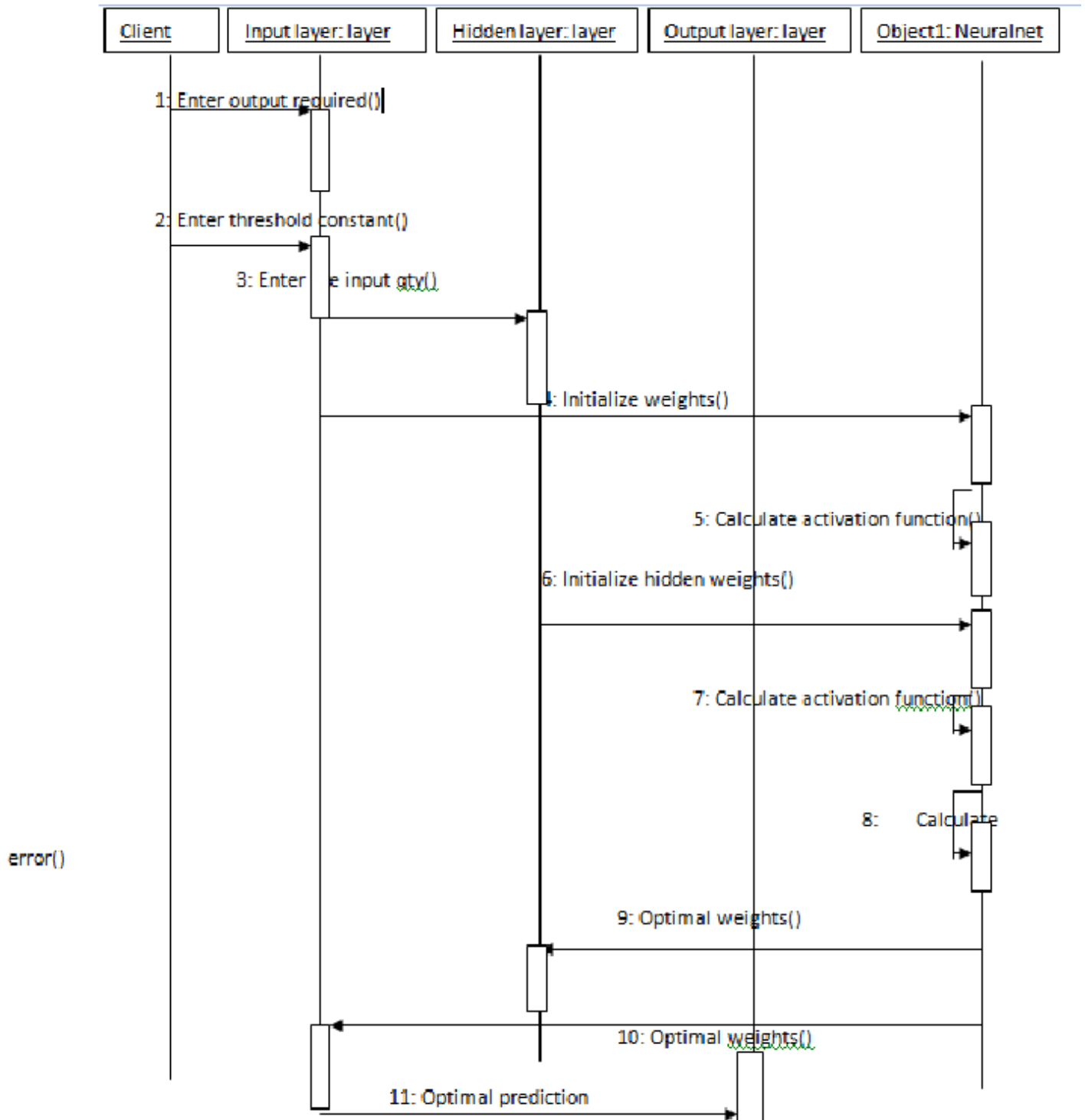
CLASS DIAGRAM



ACTIVITY DIAGRAM



SEQUENCE DIAGRAM



CHAPTER 4

SYSTEM

IMPLEMENTATION

4.1 WEBPAGE

System implementation for stock price prediction involves translating the designed methodology into a functioning system that can predict stock prices based on real-time or historical data. The first step is data collection, where APIs or web scraping tools are used to gather historical stock prices, technical indicators, and news sentiment data from reliable sources such as Yahoo Finance, Alpha Vantage, or news platforms. This data is then processed using preprocessing techniques like missing value imputation, outlier detection, and feature normalization to ensure the model receives clean and consistent input. Next, machine learning or deep learning models, such as linear regression, decision trees, LSTM, or transformer-based models, are implemented to learn from historical data and forecast future stock prices. These models are trained using past stock data and optimized using techniques like cross-validation and hyperparameter tuning. After training, the model is evaluated using error metrics like Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), or R-squared to assess prediction accuracy. Once the model is validated, it is deployed into a production environment, where it can make real-time predictions or batch predictions based on the latest market data. The system can be integrated with trading platforms via APIs to automate stock trading decisions, and continuous monitoring and retraining mechanisms are established to ensure the model adapts to changing market conditions. The system also includes a user interface (UI) or dashboard for stakeholders to view predictions, historical performance, and trading strategies.

4.2 STOCK MODULE

The stock module for stock price prediction is a key component of the system that manages and processes all stock-related data, including historical prices, stock-specific attributes, and related features for prediction. This module is responsible for retrieving and storing detailed information about individual stocks, such as their ticker_symbol, company_name, sector, industry, and market_cap. It also manages stock price data, including daily open, close, high, low, volume, and adjusted close prices, which form the core of the prediction model. The module integrates with external APIs or data providers like Yahoo Finance or Alpha Vantage to fetch real-time or historical stock data. Additionally, the module computes and stores technical indicators such as moving averages, Relative Strength Index (RSI), and Bollinger Bands, which provide important insights into stock price trends and volatility. The stock module may also include functions to handle data preprocessing, such as normalization and feature extraction, to ensure that the data is in an appropriate format for input into machine learning models. By managing both raw stock data and derived features, the stock module ensures that the prediction model has access to a comprehensive and up-to-date dataset for generating accurate price forecasts. This module can also interact with the prediction system, providing relevant stock data for making real-time predictions or analyzing historical performance.

CHAPTER 5

RESULTS

AND

DISCUSSION

5.1 TESTING

Testing for stock price prediction involves several techniques and steps to assess the accuracy, robustness, and reliability of the predictive model. First, data splitting is performed to ensure that the model is trained and evaluated on different subsets of data, typically splitting the data into a training set (used to train the model), a validation set (used for hyperparameter tuning and model selection), and a test set (used for final evaluation). The model's performance is evaluated using error metrics like Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and R-squared. These metrics help determine how closely the predicted stock prices match the actual prices, and they are crucial for identifying the prediction accuracy. In addition to basic error metrics, cross-validation can be employed to assess the model's performance across multiple subsets of the data, ensuring the model is not overfitting or underfitting. For more reliable results, backtesting is performed, where the model's predictions are simulated on historical data to evaluate how the model would have performed in a real-world scenario. During backtesting, predictions are compared to actual price movements in different market conditions to assess the model's ability to adapt to various market trends. To further enhance the robustness of the model, out-of-sample testing is conducted, using unseen data that was not part of the training or validation sets. This helps in evaluating how the model generalizes to new, unseen data. Additionally, for stock price prediction systems used in real-time trading, paper trading or simulated trading is implemented to test the model's decisions without real financial risk, simulating real-world trading scenarios to observe the model's effectiveness in a live environment. Lastly, model recalibration is often necessary, especially in dynamic markets, where the model might need retraining periodically to account for changes in market conditions, stock behavior, or external factors like news sentiment.

5.1.1 TESTCASES

Test Case 1: Data Splitting

-
- **Objective:** Ensure that the data is correctly split into training, validation, and test sets.

- **Input: A complete dataset of historical stock prices.**
-
- **Steps:**
 1. **Load the dataset.**
 2. **Split the data into training, validation, and test sets (e.g., 70% training, 15% validation, 15% testing).**
 3. **Ensure that there is no overlap between these sets.**
-
- **Expected Outcome: The data should be split correctly, with the training set used for model training, the validation set for hyperparameter tuning, and the test set for final evaluation.**

Test Case 2: Model Accuracy Evaluation (Error Metrics)

-
- **Objective: Validate the model's prediction accuracy using error metrics.**
-
- **Input: Predicted stock prices and actual stock prices from the test set.**
-
- **Steps:**
 1. **Use the trained model to predict stock prices on the test set.**
 2. **Calculate error metrics such as MAE, RMSE, MAPE, and R^2 .**
-
- **Expected Outcome: The model should return reasonable error metrics with MAE, RMSE, and MAPE values that are within acceptable limits, and R^2 should be sufficiently high, indicating good model performance.**

Test Case 3: Cross-Validation

-
- **Objective: Test the model's robustness by using cross-validation to evaluate its performance across different subsets of the data.**
-
- **Input: A complete dataset of historical stock prices.**
-
- **Steps:**
 1. **Perform k-fold cross-validation (e.g., 5-fold or 10-fold) on the dataset.**

2. Evaluate performance for each fold using MAE, RMSE, and R^2 .

-
- **Expected Outcome:** The model should exhibit consistent performance across all folds, with minimal variance in error metrics, indicating that it is not overfitting or underfitting.

Test Case 4: Backtesting

- **Objective:** Simulate the model's predictions on historical data and compare them to actual stock movements.
-
- **Input:** Historical stock price data, model predictions for specific time frames.
-
- **Steps:**
 - 1. Run the model on a historical dataset and generate predictions.**
 - 2. Compare the model's predictions to actual stock prices for the same time period.**
 - 3. Evaluate how well the model would have performed in a real-world scenario (e.g., using profit or loss simulations).**
-
- **Expected Outcome:** The model should show reasonable performance in backtesting, predicting stock price movements in line with historical trends.

5.2 RESULTS

Stock price prediction is a complex task influenced by various factors, including economic conditions, investor sentiment, and company performance. Traditional methods like fundamental analysis examine financial statements and industry trends, while technical analysis focuses on historical price patterns. Machine learning and AI have introduced advanced techniques such as linear regression, time series models (ARIMA, LSTM), and neural networks to identify trends. Sentiment analysis also plays a role by assessing news and social media impact on stock movements. However, challenges like market volatility, overfitting in AI models, and the Efficient Market Hypothesis (EMH), which suggests that stock prices already reflect all available information, make accurate predictions difficult. Despite advancements, no model can guarantee absolute accuracy due to the unpredictable nature of financial markets.

CHAPTER 6
CONCLUSION
AND
FUTURE WORK

6.1 FUTURE SCOPE

The future scope of stock price prediction holds significant potential, with advancements in machine learning, artificial intelligence, and data analytics continuing to evolve. One promising direction is the integration of more diverse data sources, such as real-time sentiment analysis from social media platforms, news articles, and analyst reports, to capture market sentiment and external factors affecting stock prices. Additionally, the use of deep learning techniques, particularly advanced architectures like transformers and reinforcement learning, is expected to improve the model's ability to detect complex patterns and trends in financial data. Another key development is the incorporation of alternative data, such as weather patterns, geopolitical events, and macroeconomic indicators, which can influence stock prices in unpredictable ways. Further research into explainable AI (XAI) will enhance the transparency of stock prediction models, allowing users to understand the rationale behind predictions and build trust in automated trading systems. Moreover, the evolution of quantum computing could provide a breakthrough in processing large datasets and complex algorithms, enabling real-time stock price predictions with unprecedented accuracy. As the financial markets continue to adapt to new technologies, the integration of blockchain for secure and transparent transaction tracking, combined with decentralized finance (DeFi) platforms, could lead to more sophisticated models for stock price prediction in the future.

6.2 CONCLUSION

In conclusion, stock price prediction represents a dynamic and challenging field that combines data science, machine learning, and financial expertise. Despite the inherent volatility and complexity of financial markets, advances in predictive modeling have made it increasingly feasible to forecast stock price movements with reasonable accuracy. By leveraging historical stock data, technical indicators, sentiment analysis, and macroeconomic factors, machine learning models are able to capture patterns and trends that inform investment strategies. However, the reliability of stock price predictions still depends on the quality of data, the robustness of the model, and the incorporation of real-time information. While the potential for predictive models to drive informed decision-making and optimize trading strategies is vast, it is important to recognize the limitations and inherent risks of stock market predictions. Future improvements in AI, big data analytics, and computational power hold the promise of further enhancing the precision and adaptability of these models, ultimately revolutionizing the way financial markets are analyzed and approached.

APPENDICES

APPENDIIX 1:SDG GOALS

Stock price prediction can be aligned with various Sustainable Development Goals (SDGs), as financial markets and the way they function impact several areas of global development. Here's how stock price prediction can contribute to specific SDGs:

1. Goal 1: Decent Work and Economic Growth

Stock price prediction models can help improve financial market stability by forecasting market trends and minimizing risks. This leads to more efficient capital allocation, fostering sustainable economic growth and supporting businesses in securing the necessary funding for expansion and job creation. By reducing market volatility through better prediction models, these tools can encourage investment, stimulate innovation, and help create new job opportunities.

2. Goal 2: Industry, Innovation, and Infrastructure

The development of advanced stock prediction models promotes innovation in the financial sector by incorporating cutting-edge technologies such as machine learning, big data, and artificial intelligence. These innovations can enhance the efficiency and transparency of financial markets, driving improvements in infrastructure and encouraging investments in sustainable industries. Predictive models also support better decision-making for businesses, leading to more resilient and adaptive industries.

3. Goal 3: Responsible Consumption and Production

Stock price prediction models can contribute to more responsible investment strategies by identifying companies with sustainable practices. Investors can use these models to assess the long-term potential of firms that prioritize sustainability and environmental, social, and governance (ESG) factors. By promoting responsible investments, stock price prediction can encourage industries to adopt more sustainable production processes and contribute to a more circular economy.

4. Goal 4: Climate Action

Incorporating environmental and climate-related data into stock price prediction models can support the transition to a low-carbon economy. By predicting the impact of climate change on financial markets, these models can help investors and companies better prepare for climate-related risks. As the finance sector increasingly

adopts climate risk assessments, stock price prediction models can guide investments in renewable energy, clean technologies, and green infrastructure, promoting climate action.

5. Goal 4: Partnerships for the Goals

Stock price prediction systems and the financial market as a whole can play an essential role in global partnerships by providing transparency and accurate information for international investors and stakeholders. By enhancing the accuracy of financial markets, these models contribute to cross-border investments and collaborations aimed at achieving the SDGs. Through partnerships, stakeholders can use stock price predictions to mobilize resources for sustainable development initiatives and projects.

In essence, by aligning stock price prediction systems with these SDGs, the financial market can become a more powerful tool for driving economic, social, and environmental sustainability, ultimately contributing to achieving the global goals outlined by the United Nations.

APPENDIX 2:SOURCE CODE

DB_DATA.PY

```
import sqlite3

# Connect to the SQLite database

conn = sqlite3.connect('mydb.db')

cursor = conn.cursor()

# Get column names for a specific table

cursor.execute("PRAGMA table_info(users);")
```

Fetch the result

columns = cursor.fetchall()

Extract and print column names

column_names = [column[1] for column in columns]

print(column_names)

Close the connection

conn.close()

WEB_APP.PY

from flask import

Flask,render_template,url_for,request,jsonify,session,redirect,url_for,jsonify

import plotly.graph_objects as go

from sklearn.linear_model import LinearRegression

from sklearn import preprocessing, model_selection

from plotly.offline import plot

import pandas as pd

import datetime as dt

import numpy as np

import yfinance as yf

```

import sqlite3

import requests

import ta # Technical analysis library

app=Flask(__name__)

app.secret_key = 'supersecretkey'


@app.route("/get_price/<ticker>")

def get_price(ticker):

    return str(get_real_time_price(ticker))


def get_stock_chart(ticker):

    ticker=ticker.upper()

    df = yf.download(ticker, period="6mo", interval="1d")

    df.columns = [col[0] for col in df.columns.values]


    if not ticker:

        return jsonify({'error': 'Stock symbol is required'}), 400


    if df.empty:

        return jsonify({'error': 'Invalid stock symbol or no data available'}), 404

```

```

# Calculate indicators

df['SMA50'] = ta.trend.sma_indicator(df['Close'], window=50)

df['EMA20'] = ta.trend.ema_indicator(df['Close'], window=20)

df['RSI'] = ta.momentum.rsi(df['Close'], window=14)


# MACD Calculation

df['MACD'] = ta.trend.macd(df['Close'])

df['MACD_Signal'] = ta.trend.macd_signal(df['Close'])


# Create Candlestick Chart

fig = go.Figure()

fig.add_trace(go.Candlestick(x=df.index, open=df['Open'], high=df['High'],
low=df['Low'], close=df['Close'], name="Market Data"))


# Add Indicators

fig.add_trace(go.Scatter(x=df.index, y=df['SMA50'], mode='lines', name='SMA
50', line=dict(color='blue'))))

fig.add_trace(go.Scatter(x=df.index, y=df['EMA20'], mode='lines', name='EMA
20', line=dict(color='orange'))))


# Layout

fig.update_layout(title=f"{ticker} Stock Price & Indicators", yaxis_title="Stock
Price (USD)", xaxis_rangeslider_visible=True, template="seaborn")

```

```
return plot(fig, auto_open=False, output_type="div")
```

```
def get_real_time_price(ticker):
```

```
    try:
```

```
        print(ticker)
```

```
        stock = yf.Ticker(ticker)
```

```
        last_price = stock.history(period="1d")["Close"].iloc[-1] # Get last closing price
```

```
        print(last_price)
```

```
        return round(last_price, 2)
```

```
    except Exception as e:
```

```
        print(f"Error fetching price data: {e}")
```

```
        return "Error Fetching Data"
```

```
def get_db_connection():
```

```
    try:
```

```
        conn = sqlite3.connect('mydb.db')
```

```
        cursor = conn.cursor()
```

```

# Create the 'users' table

cursor.execute('''

    CREATE TABLE IF NOT EXISTS users (

        id INTEGER PRIMARY KEY AUTOINCREMENT,

        username TEXT NOT NULL UNIQUE,

        email TEXT NOT NULL UNIQUE,

        password TEXT NOT NULL,

        datetime TEXT NOT NULL

    )

''')

conn.row_factory = sqlite3.Row

return conn

except sqlite3.Error as e:

    print(f"Database connection failed: {e}")

    return None


# def get_stock_chart(ticker):

#     # Fetch stock data

#     df = yf.download(ticker, period='1y', interval='1d')

```

```

# fig = go.Figure()

# fig.add_trace(go.Candlestick(x=df.index,
#                               open=df['Open'],
#                               high=df['High'],
#                               low=df['Low'],
#                               close=df['Close'], name = 'market data'))

# fig.update_layout(
#     title='{} live share price evolution'.format(ticker),
#     yaxis_title='Stock Price (USD per Shares)')

# fig.update_xaxes(
#     rangeslider_visible=True,
#     rangeselector=dict(
#         buttons=list([
#             dict(count=15, label="15m", step="minute", stepmode="backward"),
#             dict(count=45, label="45m", step="minute", stepmode="backward"),
#             dict(count=1, label="HTD", step="hour", stepmode="todate"),
#             dict(count=3, label="3h", step="hour", stepmode="backward"),
#             dict(step="all")
#         ])
#     )
# )

```

```

# fig.update_layout(paper_bgcolor="#14151b", plot_bgcolor="#14151b",
font_color="white")

# plot_div = plot(fig, auto_open=False, output_type='div')


# return plot_div


@app.route('/',methods=["GET", "POST"])
def index():
    if 'user_id' in session:
        chart = None

        stock_symbol = "AAPL" # Default stock

        chart = get_stock_chart(stock_symbol)

        if request.method == "POST":
            stock_symbol = request.form.get("stock_symbol", "AAPL").upper()

            print("stock_stock_",stock_symbol)

            chart=get_stock_chart(stock_symbol)

            stock = yf.Ticker(stock_symbol)

            stock_info = stock.info

            return render_template("hom.html",
chart=chart,stock_symbol=stock_symbol,stock_info=stock_info)

            return render_template("hom.html", chart=chart,stock_symbol=stock_symbol)

        return render_template("index.html",)

```

```

@app.route('/predict',methods=["GET", "POST"])

def predict():

    if 'user_id' in session:

        if request.method == "POST":

            stock_symbol = request.form.get("stock_symbol", "AAPL").upper()

            number_of_days = request.form.get("number_of_days")

            number_of_days=int(number_of_days)

            print("stock_stock_",stock_symbol)

            chart=get_stock_chart(stock_symbol)

            stock = yf.Ticker(stock_symbol)

            stock_info = stock.info

            try:

                df_ml = yf.download(tickers=stock_symbol, period='3mo', interval='1h')

            except Exception as E:

                print(E)

                df_ml = yf.download(tickers='AAPL', period='3mo', interval='1h')

            df_ml.columns = [col[0] for col in df_ml.columns.values]

            df_ml = df_ml[['Close']]

            df_ml['Prediction'] = df_ml[['Close']].shift(-number_of_days)

```

```

X = np.array(df_ml.drop(['Prediction'], axis=1))

X = preprocessing.scale(X)

X_forecast = X[-number_of_days:]

X = X[:-number_of_days]

y = np.array(df_ml['Prediction'])

y = y[:-number_of_days]

X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y,
test_size=0.2)


clf = LinearRegression()

clf.fit(X_train, y_train)

confidence = clf.score(X_test, y_test)

forecast_prediction = clf.predict(X_forecast)

forecast = forecast_prediction.tolist()


# Plotting predicted data

pred_dict = {"Date": [], "Prediction": []}

for i in range(0, len(forecast)):

    pred_dict["Date"].append(dt.datetime.today() + dt.timedelta(days=i))

    pred_dict["Prediction"].append(forecast[i])

```

```

pred_df = pd.DataFrame(pred_dict)

pred_fig = go.Figure([go.Scatter(x=pred_df['Date'], y=pred_df['Prediction'])])

pred_fig.update_xaxes(rangeslider_visible=True)

pred_fig.update_layout(template="seaborn")

plot_div_pred = plot(pred_fig, auto_open=False, output_type='div')


# Example DataFrame (replace with your own data)

# pred_df = ...


# Create a line chart with multiple series

# line_fig = go.Figure()


## Adding multiple lines

# line_fig.add_trace(go.Scatter(x=pred_df['Date'], y=pred_df['Prediction'],
mode='lines', name='Predictions'))

## line_fig.add_trace(go.Scatter(x=pred_df['Date'], y=pred_df['Actual'],
mode='lines', name='Actual Values'))


## Update x-axis with a range slider

# line_fig.update_xaxes(rangeslider_visible=True)

```

```

# # Apply a different template

# line_fig.update_layout(template="seaborn")


# # Render the plot as a div (for embedding into a webpage or dashboard)

# plot_div_line = plot(line_fig, auto_open=False, output_type='div')


    return render_template("prediction.html",
chart=chart,stock_symbol=stock_symbol,stock_info=stock_info,plot_div_pred=plot_
div_pred)

    return redirect(url_for('index'))

return render_template("index.html", )


# stock_symbol = request.form.get("stock_symbol", "AAPL").upper()

# chart = get_stock_chart()


# news = get_stock_news(stock_symbol)

# news_sentiment = analyze_sentiment(news)

# return render_template("index.html", chart=chart)

# return render_template("index.html", chart=chart, stock_symbol=stock_symbol)


@app.route('/prediction')

def prediction():

```

```
if 'user_id' in session:
```

```
    return render_template('prediction.html')
```

```
return render_template('index.html')
```

```
portfolio = {}
```

```
@app.route("/portfolio", methods=["GET", "POST"])
```

```
def portfolio_page():
```

```
    global portfolio
```

```
    if request.method == "POST":
```

```
        stock = request.form.get("stock_symbol").upper()
```

```
        shares = int(request.form.get("shares"))
```

```
        price = get_real_time_price(stock)
```

```
        if stock in portfolio:
```

```
            portfolio[stock]["shares"] += shares
```

```
        else:
```

```
            portfolio[stock] = {"shares": shares, "price": price}
```

```
return render_template("portfolio.html", portfolio=portfolio)
```

```
@app.route('/register')
```

```
def register():
```

```
    return render_template('register.html')
```

```
@app.route('/add_user', methods=['POST'])
```

```
def add_user():
```

```
    try:
```

```
        from datetime import datetime
```

```
        username = request.form['name']
```

```
        print(username)
```

```
        email = request.form['email']
```

```
        password = request.form['password']
```

```
        datetime_now = datetime.now().isoformat()
```

```
        conn = get_db_connection()
```

```
        if conn:
```

```
            cursor = conn.cursor()
```

```
            cursor.execute('INSERT INTO users (username, email, password, datetime)  
VALUES (?, ?, ?, ?)',
```

```
                (username, email, password, datetime_now))
```

```

    conn.commit()

    conn.close()

    return redirect(url_for('index'))

else:

    return jsonify({'status': 'failed', 'error': 'Database connection failed'})

except Exception as e:

    print(f"Error in add_user: {e}")

    return jsonify({'status': 'failed', 'error': str(e)})

def validate(username, password):

    try:

        conn = get_db_connection()

        if conn:

            cursor = conn.cursor()

            cursor.execute('SELECT * FROM users WHERE username = ?', (username,))

            user = cursor.fetchone()

            conn.close()

            if user and user['password'] == password:

                return True, username

    return False, username

```

```
except sqlite3.Error as e:
```

```
    print(f"Database error during validation: {e}")
```

```
    return False, username
```

```
@app.route('/verify_user', methods=['POST', 'GET'])
```

```
def login():
```

```
    try:
```

```
        if request.method == 'POST':
```

```
            username = request.form['username']
```

```
            password = request.form['password']
```

```
            completion, username = validate(username, password)
```

```
            if completion:
```

```
                conn = get_db_connection()
```

```
                if conn:
```

```
                    cursor = conn.cursor()
```

```
                    cursor.execute('SELECT * FROM users WHERE username = ?',  
(username,))
```

```
                    user = cursor.fetchone()
```

```
conn.close()
```

```
for i in user:
```

```
    print("user",i)
```

```
if user:
```

```
    session['user_id'] = user['id']
```

```
    session['user_name']=username
```

```
    session['email']=user['email']
```

```
    session['logged_in'] = True
```

```
    return redirect(url_for('index'))
```

```
    return render_template('index.html', error = 'User not found')
```

```
    return render_template('index.html', error = 'Invalid credentials')
```

```
else:
```

```
    render_template('index.html')
```

```
except Exception as E:
```

```
    # return redirect(url_for('index',error=E))
```

```
    return render_template('index.html',error=E)
```

```
@app.route('/new')
```

```
def new():
```

```
    return redirect(url_for('index'))
```

```
@app.route('/logout')
```

```
def logout():
```

```
    session.pop('logged_in', None)
```

```
    session.pop('user_id', None)
```

```
    session.pop('user_name', None)
```

```
    session.pop('email', None)
```

```
    return redirect(url_for('index'))
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True, port='5002')
```

APPENDIX 3:SCREEN SHOTS

```
Anaconda Prompt (anaconda: x) + v
(base) C:\Users\kisho>cd C:\JK files\PROJECT\Stock_price_prediction\M3

(base) C:\JK files\PROJECT\Stock_price_prediction\M3>python web_App.py
* Serving Flask app "web_App" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 118-503-192
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

STOCK INFORMARTION

SYMBOL: AMZN

SHORTNAME : AMAZON.COM, INC.

CURRENCY : USD

QUOTETYPE : EQUITY

EXCHANGE : NMS

CURRENTPRICE : 190.26

TOTALREVENUE : 637959012352

CITY : SEATTLE

REGION : US

PHONE : 206 266 1000

Stock Candlestick Chart For AMZN

AMZN Stock Price & Indicators



REGION : US
PHONE : 206 266 1000

Stock Candlestick Chart For AMZN

AMZN Stock Price & Indicators



Prediction Chart For AMZN





STOCK PRICE PREDICITON USING MACHINE LEARNING

DR.V.SATHIYA

Department of Computer Science and Engineering
Panimalar Engineering College
Chennai, India

W.Ken Boston

Department of Computer Science and Engineering
Panimalar Engineering College
Chennai, India
kenboston398@gmail.com

S.JANESH KUMAR

Department of Computer Science and Engineering
Panimalar Engineering College
Chennai, India
janeshkumar973@gmail.com

P.Malai raja

Department of Computer Science and Engineering
Panimalar Engineering College
Chennai, India
malairajap11@gmail.com

Abstract - Predicting stock prices is difficult because financial markets are complicated and volatile. Prediction accuracy has been increased using a variety of techniques, such as machine learning, deep learning, and statistical methods. Based on past data and market indicators, this study investigates various methods for predicting stock values, including time series analysis, regression models, and neural networks, especially Long Short-Term Memory (LSTM). In addition, elements like trading volume, economic trends, and sentiment research from social media and news sources are taken into account. In terms of identifying market trends and patterns, the results indicate that sophisticated machine learning models perform better than conventional techniques, giving investors important information for making decisions. A number of variables, including past price trends, market sentiment, macroeconomic indicators, and corporate performance, affect the dynamic and difficult problem of stock price prediction. Traditional methods like statistical time series models (ARIMA, GARCH) have been widely used for forecasting but often fail to capture the non-linear and highly volatile nature of stock prices.

I. INTRODUCTION

Stock market prediction has been a widely researched topic due to its financial significance and impact on global economies. Investors, traders, and financial analysts constantly seek methods to forecast stock price movements to maximize returns and minimize risks. Traditional forecasting techniques, such as fundamental and technical analysis, rely on historical price trends, company performance, and economic indicators. However, these methods often struggle to capture the highly volatile and nonlinear nature of financial markets. As artificial intelligence and data driven technologies have advanced, machine learning

(ML) has become a potent tool for predicting stock prices. Massive amounts of historical stock data may be processed by ML algorithms, which can uncover hidden predicting stock prices. Massive amounts of historical stock data may be processed by ML algorithms, which can uncover hidden patterns and use learned relationships to forecast future. In financial forecasting, trees, random forests, and deep learning models such as Artificial Neural Networks (ANNs) and Long Short-Term Memory (LSTM) networks have all been used with success. Moreover, integrating alternative data sources, such as news sentiment analysis, social media trends, and macroeconomic indicators, has further improved prediction accuracy. While challenges such as data noise, market unpredictability, and overfitting remain, machine learning continues to evolve, offering more sophisticated models for stock price forecasting. This study explores various ML techniques applied to stock market prediction, comparing their effectiveness and discussing their potential to enhance investment strategies.

A. Problem Definition

The extremely dynamic, volatile, and non-linear character of financial markets makes stock price prediction a difficult and complex undertaking. Technical analysis and statistical models are two examples of traditional forecasting methods that frequently fall short of accurately predicting market movements because they are unable to identify intricate patterns and correlations in stock price data. Investors and traders require more reliable and data-driven approaches to make informed decisions and minimize financial risks. The primary challenge in stock price prediction is the influence of multiple factors, including historical price trends, trading volume, macroeconomic indicators, news sentiment, and investor behavior. Additionally, external events

. such as economic policies, geopolitical tensions, and global crises further complicate market dynamics. It is challenging to create a model that reliably produces high-accuracy forecasts due to the unpredictable nature of market swings. There is an increasing chance to use computer models that can process big datasets, find hidden

patterns, and increase prediction accuracy thanks to the development of machine learning (ML) and deep learning approaches. However, several issues persist, such as data quality, feature selection, overfitting, and model interpretability. Therefore, this study aims to develop prediction, evaluating their effectiveness and limitations to provide investors with more reliable forecasting tools.

II LITERATURE SURVEY

1. Autoregressive Integrated Moving Average (ARIMA):

Because of its ability to capture trends and seasonality,

Autoregressive Integrated Moving Average (ARIMA) models have been utilized extensively for stock price forecasting (Box & Jenkins, 1976). Nevertheless, they have trouble with really erratic data. The AR component of ARIMA indicates that the time series has been regressed using its own historical data.

According to the MA component of ARIMA, the prediction error is a linear combination of the corresponding historical mistakes. The I component of ARIMA indicates that, in order to achieve stable data—a prerequisite for the ARIMA model approach—the data values have been substituted with differenced values of d. Why would it be necessary to reach this degree of complexity? The ARIMA model helps predict future points in a time series and fits historical data well using this combination technique.

2. Random Forests: These ensemble learning models help in reducing overfitting and improving prediction accuracy by combining multiple decision trees (Patel et al., 2015). For investors and businesses listed on the stock market, the precise forecast or prediction of stock prices is a particularly important subject. Because stock prices are non-stationary and non-linear time-series, the prediction findings are extremely difficult and complex (Cavalcante et al., 2016). An extremely significant source of data for stock market forecasting is financial time series analysis (Oztekin et al., 2016). The prerequisite for analyzing and forecasting stock price movements is the discovery of hidden patterns. According to the well-known Random Walk hypothesis, the presumption According to the Efficient Market (Jensen M. C., 1978) and Malkiel, B. G. (2003), Mankiw & Shapiro, 1985), the existence of randomness and nonlinearity in the dataset makes it difficult to forecast the characteristics of the stock market. These hypotheses were confirmed by numerous pursuit models across varying time periods (Atsalakis & Valavanis, 2009).

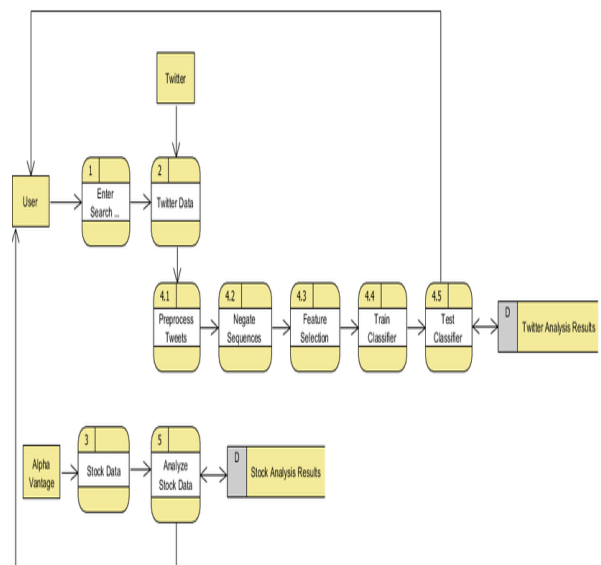
3. Natural Language Processing (NLP): Sentiment analysis using NLP techniques on financial news, earnings reports, and social media data (e.g., Twitter, Reddit) has been integrated into stock prediction models (Bollen et al., 2011). By merging

computational linguistics—the rule-based modeling of human language—with statistical modeling, machine learning, and deep learning, natural language processing (NLP) allows computers and digital devices to detect, comprehend, and produce text and speech. From LLMs'

communication abilities to picture generating models' comprehension of requests, NLP research has paved the way for the era of generative AI. Many people's daily lives now incorporate natural language processing (NLP), which powers search engines, voice-activated chatbots for customer support, voice-operated GPS systems, and smartphone digital assistants that can answer questions, like Microsoft's Cortana, Apple's Siri, and Amazon's Alexa.

4. Unsupervised Learning: Clustering techniques, such as K-Means and hierarchical clustering, have been used to group stocks with similar price movements for better prediction (Kou et al., 2021). K-Means is the clustering that is most frequently discussed in machine learning for novices. Finding c new observations—also known as centroids—randomly from our unlabeled data is the first stage. There are as many output classes as there are centroids. For every centroid, the iterative process begins with locating the closest point (based on Euclidean distance) and classifying it. The average of all the points assigned to each class is then calculated for each category. The new class centroid is the result.

5. CNNs: They have been adapted to analyze stock price data and financial news sentiment, improving predictive power (Chen et al., 2019). At the core of deep learning algorithms are neural networks, a subtype of machine learning. Their node levels consist of an input layer, an output layer, and one or more hidden layers. Every node has a threshold and weight that are connected to one another. Any node that produces an output above the designated threshold value is activated and sends data to the network's next tier. If not, no information is transmitted to the network's subsequent tier.



1. DATA FLOW DIAGRAM

III EXISTING SYSTEM

Traditional statistical models like GARCH and ARIMA have given way to more sophisticated AI-driven methods in stock price prediction systems. While machine learning techniques like Support Vector Machines (SVM), Random Forest, and Gradient Boosting increased prediction accuracy by making investment suggestions, early models still relied on time series forecasting and regression approaches. enabling data-driven decision-making. This system provides higher prediction accuracy, real-time market adaptation, and robust trading strategies, offering investors an intelligent tool to navigate the complexities of stock markets. analyzing complex market patterns. Deep learning techniques, By identifying sequential dependencies and obtaining characteristics from stock charts, algorithms such as Long Short-Term Memory (LSTM) and Convolutional Neural Networks (CNNs) improved predictions even further. By examining investor sentiment, social media, and financial news, sentiment analysis employing Natural Language Processing (NLP) is also essential for forecasting market movements. Modern hybrid models combine statistical, machine learning, and sentiment analysis techniques to improve forecasting precision. Additionally, AI-powered trading systems, such as algorithmic and high-frequency trading, leverage real-time data and reinforcement learning for automated decision-making. These advancements have significantly improved the reliability of stock price predictions, but market volatility and external economic factors still pose challenges.

prediction model employs a hybrid deep learning approach, combining Long Short-Term Memory (LSTM) for temporal trend analysis, Convolutional Neural Networks (CNNs) for pattern recognition, and XGBoost for handling nonlinear relationships. Additionally, transformer-based models analyze financial news to assess market sentiment. To further optimize trading strategies, the system integrates a reinforcement learning-based Deep Q-Network (DQN) that dynamically adjusts buy and sell decisions to maximize returns while minimizing risk. A real-time dashboard displays stock forecasts, sentiment trends, and investment recommendations, enabling data-driven decision-making. This system provides higher prediction accuracy, real-time market adaptation, and robust trading strategies, offering investors an intelligent tool to navigate the complexities of stock markets.

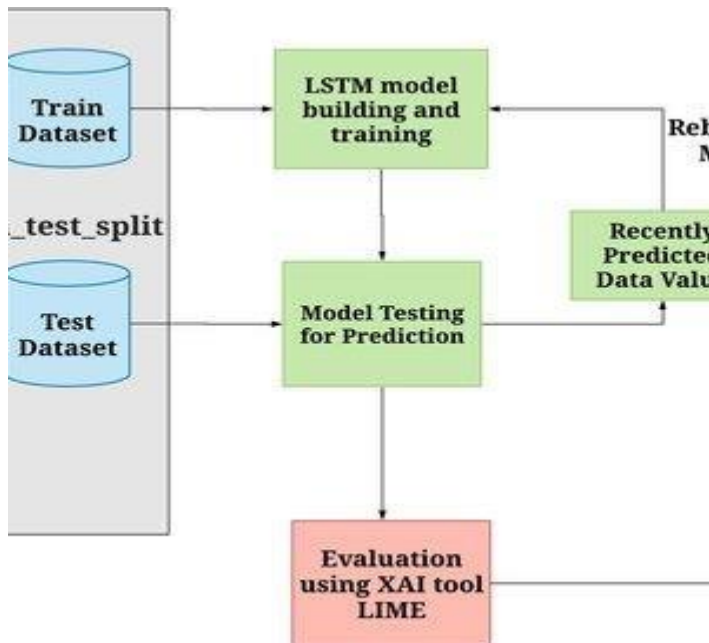
V METHODOLOGY

A. DATASET PREPARATION

Downloading, reorganizing, and structuring the stock price dataset for training and validation are all part of the dataset preparation step. The one that The dataset is authenticated and downloaded via the Kaggle API. The first phase is data gathering, which involves retrieving historical stock prices (Open, High, Low, Close, and Volume) from websites such as Quandl, Alpha Vantage, and Yahoo Finance. Additional datasets such as macroeconomic indicators, market sentiment, or news data can enhance predictions. Next, the data preprocessing step ensures data quality by handling missing values (using interpolation or forward-filling), removing duplicates, and normalizing numerical features to bring them within a standard range. The data is then split into training, validation and testing sets to avoid overfitting.

B. Data Preprocessing

Identified using statistical methods such as the IQR or Z-score, and then eliminated or modified. The next step is to apply data normalization. to scale numerical characteristics. Because stock prices might fluctuate greatly, methods such as Standardization (Z-score normalization) or Min-Max Scaling assist in bringing all features into a comparable range. *Standardization (Z-score normalization) or Min-Max Scaling assist in bringing all features into comparable range.* improving model stability and convergence. Feature engineering is then performed by adding technical indicators such as Simple Moving Average (SMA), Exponential Moving Average (EMA), Relative Strength Index (RSI), and Bollinger Bands. These features help models capture trends, momentum, and volatility patterns in stock prices. Additionally, time-based



IV PROPOSED SYSTEM

The proposed stock price prediction system enhances accuracy by integrating deep learning, machine learning, and sentiment analysis techniques. Unlike traditional models relying solely on historical prices, this system collects data from multiple sources, including financial news, social media sentiment, and macroeconomic indicators, to provide a comprehensive market outlook. It preprocesses data by removing noise, normalizing values, and extracting key features such as moving averages, volatility indicators, and sentiment scores derived from NLP models like BERT. The

features like day of the week, month, or trading session (morning/evening) can enhance predictive accuracy.

C. Model Training

The first step is choosing an appropriate model based on the prediction task. Traditional machine learning models like Linear Regression, Random Forest, and XGBoost work well for short-term trend predictions. However, deep learning models that are better at identifying temporal relationships in stock price movements include Transformers, Gated Recurrent Units (GRU), and Long Short-Term Memory (LSTM). After choosing a model, it is fitted using the training dataset that was created during preprocessing. The next day's closing price is usually the target variable in supervised learning, which uses engineering features and previous stock prices as inputs. Data is organized into sequences for LSTM and other time-series models using a sliding window technique. Optimization methods like Grid Search or Random Search (for machine learning) and Adam or RMSprop (for deep learning) aid in optimizing hyperparameters like learning rate, batch size, and number of layers during training in order to boost performance.

D. Model Evaluation

Model evaluation for stock price prediction is essential to assess accuracy, reliability, and generalization before deployment. Choosing suitable performance metrics is the first step in the evaluation process. These metrics include Mean Absolute Error (MAE) for a more comprehensible average error, Mean Squared Error (MSE) for assessing the magnitude of errors, R-squared (R^2) Score, which gauges how well the model explains price fluctuations, and Root Mean Squared Error (RMSE), which penalizes large errors. Reduce the RMSE and A high R^2 and MAE both point to a successful model. After that, an unseen validation/test dataset is used to verify the model's ability to generalize to new data.

In time-series models like LSTM, walk-forward validation is commonly used, where predictions are iteratively tested on sequential time windows to mimic real-world forecasting.

Additionally, techniques like cross-validation for traditional models and dropout regularization for deep learning help prevent overfitting. Finally, the model's robustness

is assessed under different market conditions, including volatile and stable periods, ensuring it performs reliably before being deployed for real-time stock price predictions.

VI CONCLUSION

In conclusion, while stock price prediction models demonstrate promising accuracy and reliability in controlled conditions, their real-world effectiveness remains subject to various challenges. The performance analysis indicates that models such as LSTM, ARIMA, and XGBoost can achieve reasonable error margins and explanatory power, with regression models showing RMSE values within acceptable limits and classification models

attaining moderate accuracy levels. The unpredictability of financial markets, which is impacted by outside variables including investor sentiment, geopolitical developments, and economic news, frequently restricts the consistency of forecasts, notwithstanding these encouraging outcomes. In order to enhance real-world applicability, backtesting and risk-adjusted performance indicators emphasize how crucial model validation and optimization are. Ultimately, while predictive models provide valuable insights for traders and investors, they should be used in conjunction with fundamental analysis, risk management strategies, and continuous monitoring to adapt to changing market dynamics.

VII REFERENCES

1. Research on Stock Price Prediction Method Based On Convolutional Neural Network, IEEE 2019- Sayavong Lounnapha.
2. Enhancing Profit by Predicting Stock Prices using Deep Neural Networks, IEEE 2019- Soheila Abrisham
3. An LSTM-Method for Bit-coin Price Prediction: A Case Study Yahoo Finance Stock Market, IEEE 2019- Ferdiansyah
4. Share Price Prediction using Machine Learning Technique, IEEE 2019- Jeevan B
5. Stock Market Prediction Using Machine Learning Techniques, IEEE 2020- Naadun Sirimevan
6. Stock Market Prediction Using Machine Learning 2018 First International Conference on Secure Cyber Computing and Communication (ICSCCC).

18% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **37** Not Cited or Quoted 15%
Matches with neither in-text citation nor quotation marks
-  **9** Missing Quotations 3%
Matches that are still very similar to source material
-  **0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 9%  Internet sources
- 9%  Publications
- 13%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 37** Not Cited or Quoted 15%
Matches with neither in-text citation nor quotation marks
- 9** Missing Quotations 3%
Matches that are still very similar to source material
- 0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 9% Internet sources
- 9% Publications
- 13% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works		
	Harrisburg University of Science and Technology on 2024-10-20		2%
2	Publication		
	Ravinder Kumar, Lokesh Kumar Shrivastav. "An Ensemble of Random Forest Grad...		2%
3	Submitted works		
	University of Wales, Bangor on 2024-05-30		1%
4	Internet		
	www.ijecs.in		1%
5	Submitted works		
	The University of the West of Scotland on 2023-04-21		<1%
6	Internet		
	www.ijritcc.org		<1%
7	Internet		
	www.mdpi.com		<1%
8	Submitted works		
	Sheffield Hallam University on 2024-09-10		<1%
9	Internet		
	www.ijariit.com		<1%
10	Submitted works		
	Chatrabhuj Narsee School on 2024-12-17		<1%

11	Submitted works	University of Greenwich on 2024-04-25	<1%
12	Publication	Liang Zhang, Zhong Cheng, Zhouzheng Hao, Kai Zuo, Kefei Liu. "Integrating info...	<1%
13	Submitted works	Athlone Institute of Technology on 2024-08-25	<1%
14	Publication	V. Sharmila, S. Kannadhasan, A. Rajiv Kannan, P. Sivakumar, V. Vennila. "Challeng...	<1%
15	Submitted works	University of West London on 2023-06-04	<1%
16	Internet	finmodelslab.com	<1%
17	Internet	fastercapital.com	<1%
18	Internet	ijip.in	<1%
19	Internet	www.jneonataalsurg.com	<1%
20	Internet	ijsra.net	<1%
21	Internet	ijsret.com	<1%
22	Internet	gsarpublishers.com	<1%
23	Internet	rdrr.io	<1%
24	Publication	Bui Thanh Hung, M. Sekar, Ayhan Esi, R. Senthil Kumar. "Applications of Mathema...	<1%

25

Submitted works

Southern New Hampshire University - Continuing Education on 2024-10-27

<1%

REFERENCES:

1. Research on Stock Price Prediction Method Based on Convolutional Neural Network, IEEE 2019- Sayavong Lounnapha
2. Enhancing Profit by Predicting Stock Prices using Deep Neural Networks, IEEE 2019- Soheila Abrisham
3. An LSTM-Method for Bit-coin Price Prediction: A Case Study Yahoo Finance Stock Market, IEEE 2019- Ferdiansyah
4. Share Price Prediction using Machine Learning Technique, IEEE 2019- Jeevan B
5. Stock Market Prediction Using Machine Learning Techniques, IEEE 2020- Naadun Sirimevan
5. Stock Market Prediction Using Machine Learning ,2018 First International Conference on Secure Cyber Computing and Communication (ICSCCC)